

The Next UnitTestDesign.jl

A synthesis of the Astra, Fable, and Opus proposals, with questions and options

Prepared for Andrew Dolgert by Claude (Opus 5.5)

September 23, 2026

Contents

How to read this document	2
1 The short version	3
1.1 What all three agree on	3
1.2 Where they genuinely differ	4
1.3 The recommendation in one paragraph	4
1.4 The options at a glance	4
2 Three documents, three temperaments	5
2.1 Astra: the contract	5
2.2 Fable: the product	6
2.3 Opus: the field study	8
2.4 Ideas that appear in only one document	9
3 Where the package stands	10
3.1 What is solid	10
3.2 What is broken or rough	10
3.3 A silent failure worth singling out	11
3.4 The state of your branch	12
4 The ideas, theme by theme	12
4.1 From a generator to a model	12
4.2 Names and the shape of a case	13
4.3 Constraints	14
4.4 The result object	18
4.5 Coverage as a measurement	19
4.6 Must-include cases	20
4.7 Strength, mixed strength, and budgets	21
4.8 Excursions	21
4.9 Engines, determinism, and stability	22
4.10 After the run: diagnosis and masking	22
4.11 Invalid inputs and negative testing	23
4.12 Partitions, generators, and property-based testing	24
4.13 Getting cases out	24

4.14	Words	25
4.15	First contact: positioning and documentation	25
4.16	AI agents as users	26
4.17	What none of the three addresses	27
5	The larger context	27
5.1	What the package is for, underneath	27
5.2	What these proposals teach about testing	27
5.3	What the evidence does and does not show	28
5.4	Contributing to the Julia community	29
5.5	What you would learn from each path	30
6	Questions the best version must answer	31
6.1	Purpose and audience	31
6.2	The model	31
6.3	Constraints	31
6.4	Results	32
6.5	Scope	32
6.6	Evidence and stewardship	32
7	A portrait of the best version	32
8	Options for moving forward	33
8.1	Option 1: Repair in place	33
8.2	Option 2: A small named core (Astra-led)	34
8.3	Option 3: The friendly product (Fable-led)	34
8.4	Option 4: The testing workbench (Opus-led)	34
8.5	Option 5: Core and satellites (the synthesis)	35
8.6	A research track, alongside any option	35
8.7	The options side by side	36
9	A recommendation	36
9.1	Decide now	36
9.2	Defer, with reasons	37
9.3	The first three weeks	37
9.4	A closing thought	37
	Appendix A: One example in four dialects	37
	Appendix B: Glossary	40
	Appendix C: References	41

How to read this document

You asked three models to propose the next interface for `UnitTestDesign.jl`. Each produced a substantial document:

- **Astra** (`interface_astra.md`) is a compact specification. It is written as a contract, deliberately limited, and ends with questions addressed to competing proposals.
- **Fable** (`interface_fable.md`) is a product proposal written against version 0.4.0. It is backed by probes of the current code and by an 80-line prototype of a new constraint layer that runs on today’s IPOG engine.
- **Opus** (`interface_opus.tex`), titled “Who Reaches for Combinatorial Testing, and What They Need From It,” is a user-centered review. It includes a thousand-problem experiment, a bug report, and a roadmap.

This document is meant to replace reading them. It does four things. It merges what the three found about the package as it stands ([Where the package stands](#)). It walks through the ideas theme by theme, showing where the proposals agree, where they differ, and what each difference is really about ([The ideas, theme by theme](#)). It steps back to the larger questions you raised: what the package is for, what building it teaches about testing, and what it means to offer it to the Julia community ([The larger context](#)). Finally, it poses the questions the best version has to answer and lays out coherent ways forward, ending with a recommendation ([Questions, Options, Recommendation](#)).

Some conventions. “Astra,” “Fable,” and “Opus” name the documents. A result marked *measured* was measured by the author named. I re-ran three checks against your current branch, and I say so wherever I report them. Names such as `TestSpace` or `coverage` are proposals, not existing API, and the code sketches will not run against today’s package.

One fact affects how much weight the agreements deserve. Opus says it did not read the other analyses, and Astra was written to stand against competitors rather than to build on them. Where the three agree, they mostly got there separately.

1 The short version

1.1 What all three agree on

The three proposals were written with different aims. Even so, they converge on a core so closely that I treat it as settled unless you have a reason to object.

1. **Give parameters names.** Each generated case becomes a `NamedTuple`. Names are what make everything else possible: readable failures, rules that refer to parameters by name, named mixed-strength groups, tables, and error messages in the user’s vocabulary.
2. **Never show user code a partial case.** The convention of passing `nothing` for an unsigned parameter goes away in the new interface. A rule sees only complete values for the parameters it mentions.
3. **Define coverage under constraints precisely.** A combination must be covered if and only if it appears in at least one complete case that satisfies every rule. Combinations that no valid case can contain are excluded and reported. This includes combinations that become impossible only because several rules act together. They are not silently dropped, and they are not left to crash the engine.
4. **Make the engines keep that promise.** Check feasibility before generating. Keep every partly built row completable. Never loop without progress.
5. **Return something that is still a vector but knows what it is.** The result iterates like today’s result, but it also carries the space, the strength, and what was excluded, and it prints a short summary of itself.

6. **Expose coverage measurement.** Let users measure the interaction coverage of *any* set of cases, including hand-written tests, and use the existing seed mechanism to complete them.
7. **Name the mixed-strength groups.** Replace `wayness = Dict(3 => [[3, 4, 5, 6]])` with something like `stronger = [(:a, :b, :c) => 3]`.
8. **Keep what works.** Today's positional calls keep working. IPOG stays the deterministic default and GND stays available. Excursions stay, but their different promise is made visible. Property-based testing is a partner to compose with, not a rival to absorb.

1.2 Where they genuinely differ

Three of the differences go deeper than naming.

- **How big should the package be?** Astra wants a small, exact core and an explicit list of things not to build. Opus wants the package to accompany the user through planning, running, diagnosing failures, and extending the suite. Fable sits in between and concentrates on the first-contact experience.
- **How should a rule be written?** Astra uses exact patterns plus functions with listed parameter names. Fable uses a macro over bare parameter names. Opus uses ordinary lambdas whose argument names are read as parameter names.
- **Should the package ever see test outcomes?** Only Opus proposes that the package see pass and fail results, so that it can identify the combination that caused a failure. Astra explicitly declines to build anything like a test runner.

1.3 The recommendation in one paragraph

Build a small core around one noun, the test space, and three verbs: generate, measure, and explain. First implement the constraint semantics that all three agree on, under today's API, because that is where users are hurt now. Then add the named interface. Put everything that depends on outcomes, files, or other packages at the edge, as pure functions, documented recipes, or package extensions. Before you call it 1.0, try the whole idea on a real Julia test suite and measure whether it finds faults. None of the three documents can answer that question from the armchair, and it is the one that matters most. The details are in [A recommendation](#).

1.4 The options at a glance

Option	In one line	Effort	Main risk
1. Repair in place	Fix constraints and GND under today's API	2–3 weeks	Leaves the positional pain and the <i>nothing</i> ambiguity
2. Small named core	Astra's contract, with names, rules, and coverage	+1–2 months	Correct but quiet; adoption rests on the docs
3. Friendly product	Option 2 plus Fable's macros, display, and planning	+1 month	A macro language to maintain; a breaking 1.0
4. Testing workbench	Option 3 plus Opus's diagnosis, exports, and runner	Several months	Scope beyond one maintainer
5. Core and satellites	Option 2's core, Option 3's best parts, Option 4's ideas at the edge	Staged	You have to keep saying no

A research track, a real case study with measured fault detection, can accompany any of these options. I recommend Option 5 together with the research track.

2 Three documents, three temperaments

Each proposal carries an implicit theory of what testing is. Astra treats a test suite as *a claim about a population*: a precise statement of which configurations are valid and which of their combinations were exercised. Fable treats testing as *a developer experience*: the right technique has to be the easy one, or nobody will use it. Opus treats testing as *an investigation*, a loop of planning, running, observing, diagnosing, and extending. All three theories are right. Reading the documents side by side is useful partly because the package has to decide how much of each theory to serve.

2.1 Astra: the contract

Astra starts from one question: “I have several choices to test together. Which cases will exercise their interactions without running every possible configuration?” It then proposes the least interface that answers that question honestly. Its central bet is stated plainly: “named choices, understandable constraints, and inspectable coverage make the existing algorithms useful with little additional work.” It says it does not assume that a larger feature set will create demand.

The everyday call passes a `NamedTuple` of choices to the familiar functions.

```
choices = (  
    T = (Float32, Float64),  
    storage = (:dense, :view),  
    n = (0, 1, 17),  
    pattern = (:zeros, :alternating, :random),  
)  
@testset "my_sum $case" for case in all_pairs(choices)  
    A = make_input(case; rng = Xoshiro(1234))  
    @test isapprox(my_sum(A), reference_sum(A))  
end
```

Each case is a `NamedTuple` with the same names and order as the choices. Astra stresses that names describe *test dimensions*, which need not correspond to function arguments. A case can describe how to build an array, a simulation, or a file.

The rest of Astra is a careful contract:

- **Space versus request.** `TestSpace(choices; rules)` holds the choices and the validity rules, and nothing else. Strength, mandatory cases, and engine choice belong to the generation call, so one space can serve a pairwise CI run and a stronger release run.
- **Value identity.** Domains are finite, nonempty, and ordered. Singleton domains are allowed. Values are matched by concrete type and `isequal`, so `Int8(1)`, `1`, and `1.0` are distinct choices, and duplicates are rejected. Internally the engines work with indices, so values need not be sortable. Astra also warns that selected mutable values are not deep-copied and advises constructing fresh fixtures for each case.
- **Two rule forms.** `forbid((format = :text, compression = :gzip); reason = "...")` forbids an exact partial assignment. `require((:rows, :columns); reason = "...")` do

rows, columns ... end states a relationship over named factors. A predicate over the complete case is an escape hatch. There is no partial-value sentinel, a predicate that returns missing is an error, and an exception thrown by a rule is reported with its arguments rather than treated as “forbidden.”

- **The meaning of coverage.** “A combination requires coverage if and only if it occurs in at least one complete case satisfying every rule.” Astra adds two warnings. Merely finding no immediate violation in a partial case is not enough, and generating unconstrained cases and then deleting forbidden rows cannot establish constrained coverage, because deleting a row can erase the only occurrence of another feasible pair.
- **Honesty about limits.** If a search limit prevents a conclusion, the package must say so. It must never label an unresolved combination infeasible or claim full coverage it has not established.
- **Inspection.** `isallowed(space, case)` checks a complete case. `explain(space, partial)` returns one of five outcomes: allowed, forbidden, completable (with a witness completion), infeasible, or unknown. An explanation may be less specific than a smallest conflicting set of rules, but “it must not invent a causal explanation.”
- **Results.** `TestCases` supports length, iteration, and indexing, retains its model and request, and converts to a plain vector with `collect`. `coverage(cases)` measures independently. Printing must not silently perform expensive verification.
- **Seeds.** Seeds are complete, validated, kept in the order given, and never silently discarded. Duplicates are kept but count once toward coverage.
- **Stronger groups** are written as `strength(3, (:T, :storage, :n))`, combined by union, and never modify the caller’s data.
- **Excursions stay visibly distinct.** Do not spell them as an engine of `all_pairs`, because that spelling suggests the covering guarantee.
- **Negative tests.** “Rules define the population for this particular test.” For a rejection test, generate the forbidden configurations on purpose and assert the documented exception.
- **A “not now” list.** No custom runner, macro language, model inference, CLI, serialization framework, distributed scheduling, automatic shrinking, or fixture catalog.

Astra ends with a standard of evidence. The strongest demonstration would take an existing expensive Julia test, express its choices and rules, measure its existing coverage, complete the missing interactions, and run the existing assertions. The evaluation should cover authoring effort, execution cost, and fault detection, compared with exhaustive loops and random selection at the same budget. “A smaller row count alone would not establish that the interface is worth adopting.”

What Astra does best. It is the most precise of the three about meaning. Only Astra specifies value identity, the `unknown` outcome, the difference between generated coverage and executed coverage, and what constraints mean for negative tests. It is also the most disciplined about scope.

What Astra leaves out. Astra reports no measurements, and it has little to say about first contact, documentation, or positioning. It is silent on what happens after a test fails. It keeps the names `n_way` and `seeds`. Its deliberately small default display is open to Opus’s main criticism of the current package, which is that it never tells the user what it promised.

2.2 Fable: the product

Fable is organized as a ladder of levels, each adding exactly one concept.

- **Level 0, positional.** The call keeps its shape but returns a typed `TestCases{Tuple{Int64,`

String, Float64}} instead of Vector{Vector{Any}}. Single-valued parameters become legal.

- **Level 1, names.** Parameters are given as `:n => [1, 2, 3]` pairs or as a `NamedTuple`, and a reusable `ParameterSpace` holds them.
- **Level 2, constraints.** There are three spellings. The first is a literal forbidden combination, `forbid(mode = :fast, solver = :lu)`. The second is an expression over bare parameter names, `@forbid mode == :fast && solver != :none`, with `$x` to pull in variables from the surrounding scope, and `@require` for rules stated positively. The third is a function whose parameter names are listed, `forbid(:mode, :solver) do mode, solver ... end`.
- **Level 3, the rest of the request.** Seeds may be partial. A general entry point, `covering(space; strength, stronger)`, replaces `all_tuples(...; n_way)`. Engines get readable knobs, such as `GND(candidates = 50)` in place of `M`. Excursions get an explicit base case and a distance.
- **Level 4, inspection.** `design_sizes` shows what each strategy would cost, coverage and `missing_interactions` audit existing cases, and `report` prints the summary and the unreachable combinations.

Fable’s central technical claim is that **the library should own constraints as data**. Each rule is evaluated once for every combination of values of the parameters it mentions, and the results are stored as a table of forbidden combinations. From then on the engines consult tables, and user code never runs during search. A backtracking completion search finds the combinations that no valid case can contain. The prototype ran on top of the existing `ipog_multi` without modifying it. For the solver example that crashes today, it produced five valid cases and the report “Constraints forbid 3 pairs directly and make 2 more unreachable.” On a 12-parameter example with four constraints, it produced 20 cases in 0.7 seconds.

Fable also makes the case for positioning. It proposes a decision table at the top of the README, explaining when to reach for covering designs, for `Iterators.product`, for excursions, and for property-based testing. It proposes a one-sentence explanation anchored to something every Julia user knows: “`Iterators.product(a, b, c)` is every combination; `all_pairs(a, b, c)` is the smallest subset of it in which every pair of values still occurs together.” And it measured how many random cases it takes to reach the same pairwise guarantee, discussed in [What the evidence does and does not show](#).

It closes with engine contracts (E1 through E8), a migration table, and a recommendation to call the result 1.0: “The return-type change is the only break that cannot be shimmed, and it is the change most worth making.”

What Fable does best. It has the most empathy for a first-time user and the most concreteness: nearly every claim is tied to a probe or to the prototype. The principle that the library owns constraints as data unlocks a great deal. With rules held as data, the package can print them in reports, serialize them for the command-line branch, and eventually hand them to a solver.

What Fable leaves out. The macro is a small second language, with rules about which identifiers are parameters and how `$` works, and that language has to be documented and maintained. Fable names the second constraint failure class, rows that become impossible to complete (its contract E3), but calls it rare. Opus’s data suggests that about a third of pairwise crashes on small constrained problems belong to it. Fable says little about what happens after a test fails. Its random-versus-designed comparison measures pair coverage, which designs guarantee by construction. That is fair as a statement of the guarantee, but weaker as evidence about finding faults.

2.3 Opus: the field study

Opus begins with the moments at which someone reaches for combinatorial testing: *the loop that exploded* (a nested loop over types and options went from seconds to an hour), *the expensive run* (each case is a cluster job), *the bug from nowhere* (a user hit a combination nobody tried), *the matrix bill* (the CI matrix costs too much or exceeds GitHub’s 256-job limit), and *the audit* (someone asks how thoroughly the options were exercised). It names where the package does not fit. Hunting for the one value that breaks a routine calls for property-based testing or fuzzing. Estimating how much each factor affects an outcome calls for design of experiments, because orthogonal arrays are balanced for estimation and covering arrays are not. Sweeps over continuous parameters call for space-filling designs.

It identifies eight kinds of users. Four of them are implied by the current documentation: the generic-code author, the simulation developer, the CI engineer, and the test-data builder. Four are not served at all: the *maintainer or auditor* of an existing suite, the *debugger* who wants to know which combination caused a failure, the *robustness tester* who needs invalid inputs tested one at a time, and the *AI coding agent*.

Its evidence is the most extensive of the three. Opus generated 1,000 random constrained problems (500 pairwise and 500 three-way) and checked every result with an independent coverage checker. Whenever IPOG returned a result, that result was valid and complete (888 of 888). IPOG crashed on 8.6% of pairwise and 13.8% of three-way problems. Every problem that contained an implicit constraint crashed. The remaining crashes were greedy dead ends: every required combination was feasible, but a partly built row could no longer be completed. On the same kind of input GND does not crash; it loops forever. Opus also found a one-token bug in GND’s scoring and measured its cost, found that a parameter whose real values include nothing silently loses coverage, and measured performance, stability under edits, and how quickly coverage accumulates in the first cases.

Its recommendation is to “turn the package from a generator that returns a list into a short conversation about a test space.” The conversation has five steps: *plan* (compare_strengths), *generate with a stated guarantee* (a TestDesign object whose summary lists the guarantee, what was left out, bonus higher-strength coverage, how much the first cases cover, and the next step), *run with names* (run_cases, record_outcomes), *diagnose* (diagnose, followups), and *extend* (extend, coverage, topup). Beyond that it proposes Invalid values, Partition values that draw a concrete value at run time, max_cases, rolling coverage across CI runs, exports to a GitHub Actions matrix, TOML, and literal Julia code, a one-page guide for AI agents, and a vocabulary change (must_include for seeds, strength for n_way). Its section on solvers lays out a four-part fix: a feasibility pass, a completeness invariant, guaranteed progress, and final validation. Opus’s prototype of the first three produced valid, complete designs for all 1,000 problems.

What Opus does best. It has the widest view of who uses a tool like this and when. It supplies hard numbers on how often the package fails, and it found a real bug, which you are fixing now. It is the only one of the three to take failure diagnosis seriously. Its framing, that the package “lacks a voice while it runs” and never says what it promised or what it left out, is the sharpest single criticism in the three documents.

What Opus leaves out. Its roadmap is measured in months of new API for a one-person project. Several features, including run_cases, outcome files, and rolling coverage, start to build a test framework. It puts the rules in the generation call (all_pairs(space; forbid = [...])) rather

than in the space, so the rules would have to be passed again to coverage and to any explanation. It reads rule scopes from lambda argument names through a Base internal. Its printed summary presumes verification work at display time, which is exactly the cost Astra warns about.

2.4 Ideas that appear in only one document

It is easy to lose the distinctive ideas in a merge, so here they are by source.

Only in Astra

- `explain` with five outcomes, including `unknown` when a search limit is reached and a witness when a partial case can be completed.
- Value identity by concrete type and `isEqual`, with duplicates rejected.
- The separation between the space (what is valid) and the request (how much to cover, which cases must appear, which engine to use).
- Constraints as “the population for this particular test,” so rejection tests draw on the same space.
- The caution about sharing mutable values between cases.
- The observation that keyword metadata such as `reason` can collide with a parameter of the same name, which argues for passing patterns as a positional `NamedTuple`.
- The evaluation standard: authoring effort, execution cost, and fault detection, against exhaustive and random selection at equal budget.

Only in Fable

- Tabulating each rule over its scope so that user code never runs during search.
- A macro language whose captured source can be printed, serialized, or handed to a solver.
- Partial seeds, which the engines already support internally.
- An explicit base case for excursions, and `distance` as their parameter.
- The table comparing random and designed suites.
- A full migration table, and the argument for making this 1.0.

Only in Opus

- The GND scoring bug.
- Failure rates on 1,000 random constrained problems, and the greedy dead-end class.
- Silent coverage loss when `nothing` is a real value.
- Personas, trigger moments, and an account of when not to use the package.
- Diagnosis, masking, and follow-up cases.
- `Invalid` and `Partition` values.
- Budgets: coverage by prefix, `max_cases`, bonus higher-strength coverage, and rolling coverage across CI runs.
- Exports to a GitHub Actions matrix, TOML, and literal Julia code.
- AI agents as a user group with specific needs.
- Stability of a design under small edits to the space.

3 Where the package stands

3.1 What is solid

The hard part is done. IPOG is fast and sound whenever it finishes. Opus’s independent checker confirmed every returned design on 888 constrained problems, and on fifteen four-valued parameters at strength four IPOG produced 958 cases in about four seconds, within 4% of the 924 reported in the IPOG paper. Mixed strength works and was verified independently. The package has two dependencies. The paper and the documentation explain the ideas generously.

The internals are also closer to the proposals than the interface suggests. The engines work with integer indices, so they never needed values to be sortable or comparable. The seed mechanism already counts seeded cases toward coverage, which is the foundation for measuring and completing existing suites. Fable measured that `ipog`, `ipog_multi`, and the greedy generator all accept a parameter with a single value, that `ipog_multi` fills in a partial seed and keeps it first, and that only the translation layer rejects these inputs. Functions that measure coverage (`test_coverage`, `tuples_in_trials`) already exist; they are simply not exported and work with indices rather than values.

3.2 What is broken or rough

The table merges the findings of Fable and Opus with three checks of my own. Every row was measured by at least one author.

Problem	What the user sees	Reported by
Two rules together make a pair impossible (IPOG)	<code>BoundsError: ... at index [0]</code>	Fable, Opus; rechecked
The same, with GND	Never returns	Fable, Opus; rechecked
A partly built row cannot be completed	<code>BoundsError</code>	Opus
A rule that compares with <code><</code> or <code>></code>	<code>MethodError: no method matching isless(::Float64, ::Nothing)</code>	Fable, Opus
A rule written naturally, <code>s != :none</code>	A feasible pair silently missing	This review
<code>nothing</code> is a real value of a parameter	A feasible pair silently missing	Opus
GND scores with <code>n_way</code> instead of <code>n_way - 1</code>	Designs 26–38% longer than IPOG	Opus; fixed on your branch
GND unseeded by default	Two identical calls give different designs	Opus
A must-include case with a typo, or partial	<code>MethodError: Cannot convert ... Nothing to ... Int64</code>	Fable, Opus
A must-include case that breaks a rule	Kept without comment	Opus
The guide’s own <code>wayness</code> example	<code>MethodError</code>	Fable, Opus
The caller’s <code>wayness</code> dictionary	Modified by the call	Fable, Opus
A <code>wayness</code> entry at the base strength, GND	<code>AssertionError</code>	Opus

Problem	What the user sees	Reported by
A parameter with one value	DomainError	Fable, Opus
Strength greater than the number of parameters	BoundsError	Fable, Opus
full_factorial on a large space	Allocates the whole result before any size check	Opus
Excursion base case is forbidden, or a value never appears	Dropped without comment	Fable, Opus
A failing case in test output	Any[1000, :newton, 0.001, true]	Opus
Adding one value to one parameter	3 of 10 original rows survive	Opus
The engines page of the manual	Says GND is shorter; it is longer	Opus

Opus’s random problems show how often the constraint failures occur.

IPOG on random constrained problems (Opus, <i>measured</i>)	Strength 2	Strength 3
Problems	500	500
Returned, and independently verified valid and complete	457	431
Returned but invalid or incomplete	0	0
Crashed with BoundsError	43 (8.6%)	69 (13.8%)
... of which contained an implicit constraint	28	65
... of which were greedy dead ends	15	4
Prototype with completion checks: valid and complete	500	500

These problems are small and fairly tightly constrained, so the rates are not a forecast for real projects. Opus’s three-parameter example shows that the failure is easy to hit anyway:

```
os = [:linux, :mac, :windows]; gpu = [false, true]
driver = [:cuda, :none]
dis(o, g, d) = (g === true && d === :none) ||
              (o === :windows && d === :cuda)
all_pairs(os, gpu, driver; disallow = dis)
# BoundsError: attempt to access 2-element Vector{Symbol} at index [0]
```

Neither rule mentions Windows together with a GPU. But a GPU needs CUDA, and Windows forbids CUDA, so no valid case can pair `os = :windows` with `gpu = true`.

3.3 A silent failure worth singling out

One of my checks deserves its own paragraph, because it shows the most dangerous way a testing tool can fail: it gives false confidence. Take Fable’s solver example and its first rule, “fast mode has no solver,” written the way a person would write it:

```
mode = [:fast, :exact]; solver = [:none, :lu, :qr]
tol = [1e-3, 1e-6]
all_pairs(mode, solver, tol;
          disallow = (m, s, t) -> m == :fast && s != :none)
```

```
# 6 cases, no error, no warning.  
# The pair (mode = :fast, tol = 1e-6) never appears.
```

Adding the guard `s !== nothing` produces 7 cases, including `(:fast, :none, 1.0e-6)`, which is valid. Without the guard, the rule reports every partial case that has `mode = :fast` and no solver yet as forbidden, because `nothing !== :none` is true. The package therefore treats the perfectly feasible pair `(mode = :fast, tol = 1e-6)` as forbidden, and it never appears. The documentation does warn that rules must handle `nothing`. But a rule that reads correctly and runs without error, yet quietly shrinks the coverage it was supposed to guarantee, is exactly the failure that all three proposals set out to make impossible. Opus’s finding about real `nothing` values is a second path to the same silent loss.

3.4 The state of your branch

The branch `fix/gnd-match-condition` has uncommitted changes that begin Opus’s “Phase o”:

- The one-token `fix` in `most_matches_existing`, `n_way` becoming `n_way - 1`, with a unit test of the scoring and a regression test that GND needs at most 32 cases for ten four-valued parameters, pairwise, over three seeds. Opus measured that the `fix` shrinks GND’s designs by 17–35%, and that the patched GND beats IPOG when parameters have eight values (100 cases against 113).
- `allowed_argmax`, which picks the best value among those the rule allows, rather than picking a value and then discovering it is forbidden.
- A cap on attempts in the candidate loop, which raises an error when no allowed candidate can be built.

I checked the implicit-constraint example above against this branch. IPOG still raises `BoundsError`, and GND still does not return: after 75 seconds it was still running inside `n_way_coverage_filter`. That is expected. The cap bounds the inner loop, but the outer loop is still waiting to cover a pair that no valid case contains, and only a feasibility pass can end that wait. The branch is worth merging for what it does fix. It is not the constraint fix, and its commit message should not suggest that it is.

4 The ideas, theme by theme

4.1 From a generator to a model

Today the package has verbs but no noun. You call `all_pairs` with lists of values, and you receive a list of lists. Nothing in between persists, so nothing can be asked about.

All three proposals independently introduce the noun: Astra’s `TestSpace`, Fable’s `ParameterSpace`, and Opus’s `TestSpace`. This convergence is the most important structural fact in the three documents, and it has a name in the testing literature. What these objects describe is an *input parameter model*: the tester’s choice of which dimensions of the input matter and which representative values stand in for each, together with the rules about which combinations make sense. In 1988 Ostrand and Balcer published the *category-partition method*, in which a tester divides inputs into categories, partitions each category into choices, and annotates choices with constraints. They even marked some choices `[error]` or `[single]`, to be tested once rather than combined, which anticipates Opus’s `Invalid` values by almost four decades. Grindal and Offutt later argued that

building this model is the most important and least supported step in combinatorial testing. Astra even uses the word “choices.”

Once the noun exists, the verbs multiply naturally, and each one is small:

- *count* it: how many combinations, and how many are valid;
- *check* a case against it (isallowed), or *explain* a partial case (explain);
- *generate* from it at strength t , with stronger groups and must-include cases;
- *measure* any set of cases against it (coverage);
- *complete* a set of cases (generation with must-include cases);
- *compare* strategies before choosing one;
- *walk* away from a baseline (excursions);
- *diagnose* failures among its cases;
- *export* its cases.

This is why a model is also the answer to the scope question. The model is the stable thing, and the operations are optional additions that do not change it. Astra’s minimalism and Opus’s breadth stop being in tension once you see that both are lists of operations on the same object. The real questions become which operations to ship first and which to leave to users.

What belongs in the model? All three agree on names, values, and validity rules. Astra draws a line I find persuasive: the space describes *the world*, meaning which configurations are valid, while the request describes *your intent*, meaning how much to cover, which cases must appear, and which engine to use. By that test, stronger groups and must-include cases belong in the request. Several proposed additions sit on the line, and each is a small decision:

- *value labels* (Fable’s open question): “tiny” => 1e-9, so that test names and reports stay readable;
- *invalid markers* (Opus’s Invalid(-1)): these describe the world, but they change how cases are generated;
- *partitions with generators* (Opus’s Partition): a named class that draws a concrete value at run time;
- *weights*, which your feature/turing-generation branch explores as a way to bias generation toward common usage. None of the three proposes them.

A small trap in the constructor. The prettiest constructor, `TestSpace(os = [...], gpu = [...])`, cannot also accept `constraints = [...]` without reserving `constraints` as a name no parameter may have. Astra noticed the same collision in `forbid(mode = :fast; reason = "...")`: the metadata keyword and a parameter named `reason` would clash. The robust spellings are a `NamedTuple` passed positionally, `TestSpace((os = ..., gpu = ...); constraints = ...)`, and `name => values` pairs, which also allow names that are computed rather than typed. Accept both. It is a small point, but it is the kind that is impossible to fix after 1.0.

4.2 Names and the shape of a case

All three agree that each case should be a `NamedTuple`. The consequences reach further than they first appear:

- Test output becomes readable without any extra machinery: `@testset "solve $case" for case in cases` prints `(n = 1000, method = :newton, tol = 0.001, sparse = true)` instead of `Any[1000, :newton, 0.001, true]`.

- Code under test can be called by keyword (`f(; case...)`), by destructuring (`(; n, method) = case`), or by position (`f(case...)`) when the order of the names matches the function’s arguments.
- A vector of `NamedTuples` is already a `Tables.jl` row table, so `DataFrame(cases)` and `CSV.write` work without any new dependency (Fable).
- Error messages can name the parameter and list its values, which Opus argues matters especially for AI agents, since they repair code by taking messages literally.

Astra adds two points of discipline. First, names are *test dimensions*, not necessarily function arguments. A case might describe how to construct an input rather than the input itself. The manual’s partition-testing example is a good illustration: a partition `:split` maps to `(-2, 3)` in user code. Second, value identity has to be defined, and Astra’s choice (same concrete type and `isequal`) is the one that respects Julia’s type system, where `Float32` and `Float64` versions of the same number really do exercise different code.

Fable raises one issue that names do not solve. A `NamedTuple` is only as readable as its values. A matrix, a large struct, or a function prints poorly in a test name. Value labels would solve this, and so would the habit Astra recommends of choosing descriptive values (`:ill_conditioned`) and constructing the real input in the test body. The second approach needs no API at all.

There is one real disagreement about names. Fable wants the *positional* path to change too, returning a typed `TestCases{Tuple{...}}` instead of `Vector{Vector{Any}}`. Astra and Opus want the positional path left alone. The change would break only code that mutates result rows or depends on the concrete type, and it buys type stability that rarely matters in test code. I would decide this at 1.0, after looking at which registered packages depend on `UnitTestDesign` and how they use it.

4.3 Constraints

This is the heart of the next version. It is where users are hurt today, and it is where the three proposals do their deepest thinking.

4.3.1 What went wrong, conceptually

The current engines ask a *local* question: “is this partial case forbidden yet?” Coverage needs the answer to a *global* question: “can this partial case still be completed into a valid case?” The gap between those two questions produces both failure classes that Opus measured:

1. **Implicit constraints.** A combination passes every rule on its own, yet no complete valid case contains it. The engine treats it as required, cannot place it, and crashes (IPOG) or waits forever (GND).
2. **Greedy dead ends.** Every required combination is feasible, but the engine builds a row by checking only that it is not *yet* forbidden, and the row reaches a state from which no allowed value fits the remaining slot.

Anyone who has studied constraint satisfaction will recognize this as the difference between local consistency and global consistency. Rules that are each reasonable can combine into consequences that nobody wrote down. Constraint programmers call such derived consequences *nogoods*. The literature on combinatorial testing reached the same conclusion long ago. Cohen, Dwyer, and Shi (2008) put a SAT solver inside an AETG-style greedy generator. Yu and colleagues built IPOG-C (2013), which checks validity with a constraint solver and optimizes how often it has to ask, and

later proposed deriving the minimal forbidden tuples in advance (2015). NIST’s ACTS tool ships this approach. UnitTestDesign’s engines were written without this machinery, which is why they fail on precisely these cases.

The fix has four parts, and between them the proposals describe all four:

1. **A feasibility pass** (Fable’s E2, Opus’s part 1). Before generating, check every target combination for a valid completion. Drop the ones that have none, and report them.
2. **A completability invariant** (Fable’s E3, Opus’s part 2). Whenever an engine sets a value, require that the row can still be completed. In IPOG this touches three places (`choose_last_parameter_filter!`, `insert_tuple_into_tests_filter`, and `fill_remaining_missing_values_filter!`). In GND it touches candidate construction.
3. **Guaranteed progress** (Fable’s E4, Opus’s part 3). With the first two parts in place, IPOG cannot reach a dead end, and GND always has a covering candidate, because the first uncovered combination is itself completable. Every loop should still stop with a diagnosis if a pass makes no progress.
4. **Final validation** (Opus’s part 4). Check every returned row, including must-include cases, against every rule.

Fable’s prototype implemented the first part on top of IPOG, by adding the infeasible combinations to the predicate, and it fixed Fable’s examples. Opus’s prototype implemented the first three parts around a simple greedy generator and fixed all 1,000 problems. Opus’s data settles a point on which Fable was uncertain: the second part is necessary, not optional, because greedy dead ends caused 15 of the 43 pairwise crashes and they occur even when no implicit constraint exists.

What it costs. The completability check is a backtracking search over unassigned parameters. It is exponential in the worst case and cheap in practice, because real rules are local. When the scope of each rule is known, only the parameters connected to the row’s assigned parameters through shared rules need to be searched, and the rest can be filled freely. Fable’s 12-parameter example took 0.7 seconds end to end, most of it inside IPOG.

4.3.2 The semantic contract

All three proposals state the same contract, in different words. Written out in full:

1. A **complete case is valid** if and only if it satisfies every rule.
2. A t -way combination is **required** if and only if it is **feasible**, meaning that at least one valid complete case contains it.
3. Every returned case is valid, and every required combination (at the base strength and within every stronger group) appears in at least one returned case.
4. Infeasible combinations are **reported**, distinguishing those a single rule forbids from those that are impossible only because rules combine.
5. **User code never sees a partial case.** A rule is evaluated only on complete values for the parameters in its scope.
6. If a resource limit prevents a conclusion, the result says **unknown**, and never “infeasible” or “complete.”

Astra’s example is the clearest illustration of point 2. Let A, B, and C each take the values 1 and 2, with the rules $A == B$ and $B == C$. The only valid cases are (1, 1, 1) and (2, 2, 2). The pair (A = 1, C = 2) is infeasible and does not need coverage, and the user never had to write the implied rule $A == C$.

4.3.3 Five ways to write a rule

This is the most visible disagreement among the proposals, so it helps to line up the candidates.

Form	Example	Scope known by	Printable	Magic
Exact pattern (Astra, Fable)	<code>forbid((os = :mac, gpu = true))</code>	the pattern's names	Fully	None
Function with listed names (Astra, Fable)	<code>require(:rows, :cols) do r, c ... end</code>	the listed names	Names and reason	None
Lambda argument names (Opus)	<code>(os, gpu) -> os == :mac && gpu</code>	reflection on argument names	Names only	A Base internal
Macro over bare names (Fable)	<code>@forbid os == :mac && gpu</code>	free identifiers in the expression	Full source	Macro rules for names and \$
Whole-case predicate (Astra; legacy)	<code>require() do case ... end</code>	none: all parameters	reason only	None

Each form has its own character:

- **Exact patterns** are the most common real constraint (“no ARM on Windows”). They are pure data: they print, serialize, compare, and translate to a solver trivially. Astra’s positional `NamedTuple` spelling avoids the collision between keyword metadata and parameter names.
- **Functions with listed names** are the workhorse. They accept any Julia code, and their scope is explicit. The only cost is that the names appear twice, once as symbols and once as arguments.
- **Lambda argument names** are the most elegant to read. But `Base.method_argnames` is internal to Base, and the approach behaves oddly with `_` arguments, destructured arguments, varargs, and callable structs. It also turns renaming a lambda’s argument into a change of meaning, in code that does not look as though it has anything to do with parameter names. Opus itself suggests an explicit fallback, `(:os, :gpu) => (o, g) -> ...`, which is really the listed-names form.
- **The macro** is the most readable form and the only one that keeps the rule’s source for reports (“@forbid mode == :fast && solver != :none forbids 2 pairs”), for a TOML format, and for translation into a solver. Its costs are those of any small language: the convention that every bare identifier names a parameter, \$ for outside variables, errors at macro expansion, and a maintenance burden. Fable’s design handles the most common mistake well: a misspelled name produces an error that suggests writing `$solvr` if a variable was meant.
- **Whole-case predicates** are needed for legacy `disallow` and for rules that really are global (“fits the memory budget”). Their scope is every parameter, so checking feasibility may require searching large parts of the full product.

The most useful observation here is that **every form can compile to one internal representation**: a scope (a tuple of names), a predicate, a polarity (forbid or require), and a label (a reason, source text, or both). The choice of surface syntax is therefore reversible and additive, and

the choice of internal representation is not. Decide the representation now. Surface forms can be added one release at a time, according to what users ask for.

Both polarities. Astra and Fable both offer `forbid` and `require`. This is not a luxury. People state rules both ways (“exact mode *requires* a tight tolerance”; “*no* CUDA on Windows”), and translating one polarity into the other by hand is a classic source of bugs. Today’s `disallow` accepts only one polarity, and Fable counts that among its five usability defects.

4.3.4 Tabulate, evaluate, or solve

Fable proposes evaluating each scoped rule once for every combination of values in its scope, and storing the result as a table. For the rule over `(mode, solver)` that is six evaluations. The benefits are real. The number of calls to user code is bounded and known in advance. Errors in rules surface when the space is built rather than deep in the engine. The tables can be printed and serialized. Any engine, or a future solver, can consume the same tables. The cost is the product of the scope’s domains, which is small for real rules; Fable proposes a warning above a threshold.

Astra’s contract says only that predicates must be deterministic and may be evaluated more than once, in an unspecified order. Tabulation is one implementation that satisfies that contract, so the two proposals do not actually conflict. Opus evaluates lazily and caches, which for deterministic predicates gives the same answers. My recommendation is to specify Astra’s contract, implement Fable’s tabulation for scoped rules, and accept whole-case predicates at an honestly documented cost.

Where Z3 fits. All three keep solvers out of the core. Your `z3_example.jl` translates comparisons of a parameter with a literal, combined with `&&` and `||`. That is narrower than what users can write today, and it would add a large binary dependency to solve problems for which a short backtracking search is enough. Fable would allow a later package extension fed by macro-captured expressions, and Opus would allow one that answers the same completability question faster. The honest position is that backtracking over tabulated rules suffices for the sizes people bring. A solver becomes interesting only when there are many parameters *and* many interacting rules, or when whole-case rules dominate. Keeping rules as data keeps that door open at no cost.

4.3.5 Where rules live

Astra and Fable put the rules in the space. Opus passes them to the generation call, and Fable allows both. The argument for the space is decisive to me: validity is a property of the system under test, and every operation (`coverage`, `explain`, `isallowed`, `excursions`, `planning`) needs it. If rules live in the call, each of those operations needs them passed again, and a mismatch between the rules used to generate and the rules used to measure becomes possible. A one-line convenience that accepts `constraints = ...` in a generation call and builds a space internally costs nothing and keeps the one-liner.

4.3.6 Reporting what was left out

All three want the result to say what was excluded. The design question is how far to go in explaining *why*.

- *Directly forbidden* combinations are easy: a single rule forbids them, and that rule can be named.

- *Implied* combinations are harder. Opus’s display says “impossible: rules 1 and 2 together.” Finding the rules responsible means finding a small set of rules that is unsatisfiable together with the combination. Astra says the first implementation need not find a smallest such set, but “must not invent a causal explanation.” For the handful of rules real spaces contain, a simple deletion search is cheap and honest: drop one rule at a time and see whether the combination becomes feasible. So Opus’s display is attainable without cheating.

The report is valuable beyond explaining the design. Fable makes the point well: a user who did not *intend* (`solver = :lu`, `tol = 1e-3`) to be untestable learns it from the report. **The list of implied exclusions is a check on the user’s specification.** Rules that are each sensible can have consequences their author never meant, and the package is in a position to show those consequences. Few testing tools do this, and it may be among the most useful things this one can say.

4.3.7 What happens to `disallow`

Two migration paths are available, and they trade speed against safety.

- **Use `disallow` as a pruning oracle.** The documented contract of `disallow` is that it returns `true` only when a case, possibly partial, is certainly forbidden. That is exactly the question a backtracking search needs answered in order to prune. So the feasibility pass and the completeness invariant can be built on the *current* API and will be correct whenever the user’s rule honors its contract. This is what makes a correctness release possible before any new interface exists. The nothing ambiguity and the over-forbidding trap would remain.
- **Evaluate `disallow` on complete cases only** (Fable’s proposal). The rule never sees nothing, so both traps disappear. The price is that every feasibility question becomes a search over all parameters. That search is fast when a witness exists, which is the common case, but it can be slow to prove that a combination is infeasible in a large space.

A reasonable path is the first option for a correctness release under today’s API, followed by deprecating `disallow` in favor of scoped rules in the named interface.

4.3.8 Dependent parameters

The most common real constraint is that one option applies only when another has a particular value: a preconditioner applies only to iterative solvers, and a GPU driver only when there is a GPU. Fable’s recipe is a sentinel value (`:none`) plus one `require` rule. The recipe works with no special support. Its cost is that the sentinel pairs with every value of every other parameter, so the design spends rows exercising the “not applicable” branch alongside everything else. Usually that branch is worth exercising. Other tools model these trees directly (PICT, for example, supports sub-models), and a first-class notion of conditional parameters could come later if the recipe proves clumsy.

4.4 The result object

All three want a result that behaves like today’s vector and also knows what it is. They differ on the name (`TestCases` in Astra and Fable, `TestDesign` in Opus), on element typing (Fable wants `TestCases{T} <: AbstractVector{T}` with concrete `T`), on what it prints, and on whether the positional path changes.

The most interesting difference is what the display should show and what it may compute. Fable’s display leads with a single line: “10 test cases, pairwise, 4 parameters, full factorial would be 81.” In Fable’s words, “10 of 81 is the whole pitch of the package.” Opus’s display is a small report:

```
TestDesign: 10 cases over 5 parameters (strength 2, IPOG, deterministic)
  Guarantee  every feasible pair of values appears in at least one case
              55 of 55 feasible pairs covered (checked)
  Space      72 combinations in full; 48 satisfy the rules
  Left out   2 pairs forbidden by rules
  Bonus      83 of 120 feasible triples also covered (69%)
  Budget     first 5 cases cover 73% of pairs; first 8 cover 95%
  Next       strength 3 needs 20 cases: extend(design; strength = 3)
```

Astra’s warning is that printing must not quietly start expensive work. It helps to sort these lines by what they cost.

- **Free at display time.** The engine already knows its target combinations and which ones it covered, and the feasibility pass already knows what it excluded. “55 of 55 feasible pairs” and “2 pairs forbidden” are bookkeeping, not verification. So are the row count, the strength, the engine, the seed, and the size of the full product.
- **Cheap but not free.** Independent verification (“checked”), bonus coverage at a higher strength, and coverage by prefix all require counting combinations over the rows, which amounts to a few hundred thousand operations at typical sizes. These belong in an explicit report or coverage call, not in show.
- **Potentially expensive.** “48 satisfy the rules” requires counting the valid cases in the full product. That is trivial for 72 combinations and hopeless for 10^{12} . Show it only when it is cheap.

A synthesis of the two displays:

```
TestCases: 10 cases, strength 2 (IPOG)
  space      5 parameters; 72 combinations, 48 valid
  covers     all 55 feasible pairs
  excluded   2 pairs forbidden by constraints; see excluded(cases)
  (os = :linux, julia = "1.10", threads = 1, gpu = false, arch = :x64)
  (os = :linux, julia = "1.11", threads = 4, gpu = true, arch = :arm64)
  ... 8 more
```

The “next step” line in Opus’s display is a good idea, but it raises a question of tone. A result that suggests `extend(design; strength = 3)` every time it is printed may come to feel like nagging. It could appear only in report.

4.5 Coverage as a measurement

If I had to pick the single most leveraged new function, it would be `coverage(cases, space)`. It serves more users than generation does:

- **Adoption without replacement.** Most projects already have tests. A maintainer can describe the space, measure what the existing tests cover, and add only what is missing. Nobody’s tests have to be replaced, which makes this the gentlest way to bring combinatorial testing into a project. NIST built a tool, CCM, for exactly this job.

- **Audit.** It answers “how thoroughly were the options exercised?” with a number and a list.
- **Campaigns.** When half of a simulation campaign crashed, measuring the cases that actually ran shows what is still untested. Astra notes that coverage describes whatever cases are supplied, so the user should pass the cases that actually ran.
- **Checkable claims.** An agent, or a human reviewing an agent’s work, can verify the guarantee in one line rather than trusting the generator.
- **Random and property-based testing.** A developer who prefers random cases can draw them, measure their coverage, and top up the missing combinations. This makes the package useful even to people who never generate a covering design.

The three proposals present coverage workflows under different names: Fable has `missing_interactions`, Opus has `coverage`, `topup`, and `extend`, and Astra describes a workflow in three steps. **They are one mechanism seen from three sides.** Given `coverage` and `must-include` cases:

- *audit*: `coverage(existing, space)` lists what is missing;
- *top-up*: `all_pairs(space; must_include = existing)` keeps the existing cases and adds the fewest the engine can find;
- *extend*: `all_triples(space; must_include = design)` raises the strength while keeping every current case.

`topup` and `extend` can be one-line conveniences or documented recipes. They do not need new machinery, and that resolves one of the scope tensions between Astra and Opus.

A few definitions have to be settled, and Astra settles most of them. Invalid rows contribute no coverage. Duplicates count once. The report covers the requested strength and every stronger group. Resource limits produce “unknown,” never an invented percentage. In prose, the concept should be called *interaction coverage*, because “coverage” alone means line coverage to most programmers (Opus). And Opus raises a subtler point in its discussion of masking: coverage by the cases that *ran* is not the same as coverage by cases that *passed*. A combination that appeared only in a failing case has not been shown to work.

4.6 Must-include cases

Today these are called `seeds`. The proposals differ on three points.

- **The name.** Opus would rename them `must_include`, because the package also uses random-number seeds, and so does its test suite. That is a real collision in a package that has both an RNG and seed cases. Keep `seeds` as an alias.
- **Complete or partial.** Astra requires complete cases. Fable allows partial ones, which the engine already supports internally. A partial `must-include` case has an elegant reading: it is simply one more required combination, of whatever size the user wrote. A complete case is the special case in which the combination covers every parameter. On this reading, partial cases are not a new feature. They are the general form.
- **Invalid cases.** All three agree that a `must-include` case that breaks a rule should be an error, with an explanation that names the case, the parameter, and the value. Opus would allow an override “if the user insists.” Since invalid rows contribute no coverage, the only reason to insist is negative testing, which is better handled deliberately (see [Invalid inputs](#)).

Astra also specifies the order: `must-include` cases come first, in the order given, followed by the generated ones. That ordering is what makes the `top-up` workflow legible.

4.7 Strength, mixed strength, and budgets

The renaming has consensus apart from Astra, which keeps `n_way`. `strength` is the literature’s term and Fable and Opus both adopt it, keeping `n_way` as an alias. Named groups replace positional wayness. Fable’s and Opus’s `stronger = [(:julia, :threads, :gpu) => 3]` is lighter than Astra’s `extra = [strength(3, (:T, :storage, :n))]` and says the same thing. The validation rules are Astra’s: names must exist, names within a group must be distinct, a group’s strength cannot exceed its size, overlapping groups combine by union, and the caller’s data is never modified.

Choosing a strength. Users struggle with this decision more than any other, and nobody can make it for them. IPOG is fast enough to simply try each option, which is what Fable’s `design_sizes` and Opus’s `compare_strengths` both do. They are the same function. Opus’s measured example, for a CI space of five parameters and two rules:

strength	cases	share of allowed	pairs covered	triples covered
1	3	6%	49%	24%
2	10	21%	100%	69%
3	20	42%	100%	100%
4	39	81%	100%	100%

A table like this turns an abstract choice into a budget decision. It also teaches something users rarely see: a pairwise design covers a large share of triples by accident. Opus measured 25% for five four-valued parameters, and up to 61% for twenty three-valued parameters.

Budgets. Opus’s other finding is that IPOG front-loads coverage reasonably well (16 of 28 cases reach 90% of pairs), but nothing documents this and nothing exposes it. A `max_cases` option, or simply a guarantee that cases are returned in an order that accumulates coverage quickly, would serve the user who has thirty cluster runs and not three hundred. Petke and colleagues studied exactly this question of how early a combinatorial suite finds faults when run in generation order. One hypothesis is worth testing once the GND fix lands. Generators that build one row at a time, choosing each row to cover as much as possible (AETG, and GND after it), front-load coverage by construction, while IPOG’s order is incidental. If so, GND has a clear reason to exist: it would be the engine for budget-limited runs.

Rolling coverage. Opus’s most charming idea is to seed GND differently on each CI run (with the run number, say), so that each run is a valid pairwise design and higher-strength coverage accumulates across runs at no extra cost. It needs a record of which cases ran, which puts it on the far side of the question about whether the package sees outcomes.

4.8 Excursions

Excursions make a different promise from covering designs: every case lies within distance d of a base case, meaning that it differs from the base in at most d parameters. Today the base is silently the first value of every parameter, which Fable calls “the single most surprising fact about `values_excursion`.” The proposals agree on the repairs: an explicit base (`from = ...`), distance as the parameter, the old names kept as aliases, a report when the base itself is forbidden or when some value never appears, and, as Astra insists, never spelling excursions as an engine of `all_pairs`.

The deeper point is what excursions are for. They are the natural design when you have a trusted

configuration, such as production settings or a reference run, and you want to know which departures from it break things. A failure at distance 1 identifies its own cause. That makes excursions a diagnostic tool as much as a generation strategy. The next section builds on this: Opus’s “follow-up cases” after a failure turn out to be excursions from the failing case.

4.9 Engines, determinism, and stability

GND. Your branch fixes the scoring bug. Should GND stay? Fable says yes, but keep it off the first page. Opus’s measurements give it a clearer role: once patched, GND beat IPOG at eight values per parameter (100 cases against 113). It may also be the right engine for budgets and for rolling coverage. Its knob `M` would be clearer as candidates (Fable).

Determinism. `GND()` currently seeds itself from an unseeded global generator, so two identical calls differ (Opus). All three want determinism by default. Opus would give GND a fixed default seed, record it in the result, and print it when a test fails. Astra draws a boundary worth adopting as written: the default is deterministic “for a fixed model and implementation version. Exact ordering across future versions is not a promise. Explicitly saved cases are stronger regression records than a generator seed alone.”

Stability under edits. Opus found that adding one value to one of five parameters changed the IPOG design from 10 cases to 14, and only 3 of the original rows survived. For anyone who compares failures across commits, that matters. Passing the old design as must-include cases preserves it today, but nothing suggests doing so.

These three points together suggest a pattern that none of the proposals states in full, which I will call a **design lockfile**. Commit the cases to the repository as a literal, which Opus’s `as_code` would print. Add one test asserting that `coverage(cases, space)` is complete. When someone adds a value or a rule, that test fails and names the missing combinations. Regenerating with the old cases as must-include cases then preserves every existing row and adds only what is new. The pattern gives stability across edits, reproducibility across package versions, a reviewable diff of exactly which cases run, and a check that fails when the space and the cases drift apart. It needs only coverage, must-include cases, and a printer. Its analogue in package management is the relationship between `Project.toml` and `Manifest.toml`.

Performance. IPOG runs in well under a second at typical sizes. Allocation is heavy, 17 GB cumulative for fifteen parameters at strength four, because hot loops slice matrix columns. Opus suggests views or explicit loops. This is worth doing eventually, but it is not urgent. The one real hazard is `full_factorial`, which allocates before it checks the size (Opus computes 550 GB for sixteen four-valued parameters). A size guard fixes it.

4.10 After the run: diagnosis and masking

Only Opus goes here, and the idea deserves to be read in full. It starts with a solver that fails only when `method = :newton` and `sparse = true`, tested with a pairwise design of ten cases. Two cases fail. The design and the outcomes already contain the answer. About twenty lines of code find the combinations that appear in failing cases but never in passing ones:

```
suspicious: method=:newton, sparse=true      (in 2 of 2 failures, 0 of 8 passes)
suspicious: n=1000, method=:newton           (in 1 of 2 failures, 0 of 8 passes)
suspicious: n=100, sparse=true               (in 1 of 2 failures, 0 of 8 passes)
```

The true cause ranks first. The other two pairs are *masked*: they appeared only in failing cases, so nobody knows whether they work. Two follow-up cases would settle all three: (1000, :newton, 0.001, false) and (100, :gmres, 1e-6, true). Look at those follow-ups closely. Each one is an excursion: it keeps one suspicious pair and changes the value that would distinguish it from the others. Diagnosis and excursions share machinery.

The idea has deep roots. A covering design used for diagnosis resembles *group testing*, in which items are pooled, each pool is tested once, and the pattern of positive pools identifies the defective items. Colbourn and McClary (2008) defined *locating arrays* and *detecting arrays*: covering arrays with the extra property that the pass and fail pattern alone identifies a small number of faulty interactions. Ghandehari and colleagues built BEN, a tool that ranks suspicious combinations and generates follow-up tests. Yilmaz and colleagues studied *masking*, the effect by which a failure hides the behavior of every other combination in the same case, so that the coverage guarantee is weaker than it looks until the masked combinations are retested.

My reading. This is the direction with the most intellectual depth and the one in which a covering design has a special advantage, because the cases are few, structured, and contain every combination. It is also the direction that crosses Astra’s line, because the package has to see outcomes. There is a way to cross that line without building a runner: make diagnosis a pure function of the cases and their outcomes,

```
diagnose(cases, passed::AbstractVector{Bool})
```

and let users collect outcomes however they like, whether in a Test.jl loop, a cluster job, or a spreadsheet. Opus’s run_cases and record_outcomes are conveniences that can come later, or remain recipes.

Diagnosis needs careful semantics before it is presented as more than a suggestion. There may be several faults at once. Failures may be intermittent. A single value may be to blame, so diagnosis should rank single values as well as pairs. A wrong answer is not the same kind of event as a thrown error. The candidate list generates hypotheses; it does not prove anything. A first version labeled “experimental” would be honest about all of this.

4.11 Invalid inputs and negative testing

Astra and Opus address two different needs here, and both are real.

Testing that invalid configurations are rejected. Astra’s point is that “forbidden” must not come to mean “never tested.” Rules define the population for *one* test. A numerical-correctness test excludes unsupported configurations, and a rejection test deliberately generates those same configurations and asserts the documented exception. This needs no new API: a second space with different rules, or explain to confirm that a case is forbidden, is enough. What it needs is documentation that shows both uses side by side.

Avoiding error masking in robustness tests. Opus’s point is that if a case contains two invalid values, the first one to trigger an error hides the other. Microsoft’s PICT marks values as invalid and generates cases with at most one invalid value each, while still pairing every invalid value with every valid value of the other parameters. Category-partition’s [error] annotation expressed the same idea in 1988. This does need generation support, and a coverage report that states two guarantees separately.

The first need is met by documentation. The second is a real feature with its own semantics, and it could wait until the named core is stable.

4.12 Partitions, generators, and property-based testing

The paper already contains the key idea: each value in a combinatorial specification can stand for an *equivalence class*, and a generator can draw a concrete member of that class at run time. Combinatorial testing then decides *which classes to combine*, and random or property-based testing decides *which values to draw within each class*. All three proposals endorse this partnership, and they differ only in how much to build for it.

- **Astra:** nothing. A case chooses a category such as `:near_zero`, and user code draws within it. More repetitions explore more values under the same configuration coverage.
- **Fable:** document the pattern. Nothing should be automatic, because functions are legitimate test values in their own right (`[sin, cos]`), so the package cannot assume that a function-valued choice is a generator.
- **Opus:** a first-class `Partition(:tiny, rng -> 1e-12 * rand(rng))`, drawn from a recorded seed, with a possible extension that accepts `Supposition.jl` generators.

Fable’s concern is decisive for the first release. A `Partition` type would have to be opt-in and explicit, never inferred. Note also that Fable’s value labels (`"tiny" => 1e-9`) and Opus’s partitions (`:tiny => generator`) are the same idea from two directions: a *named choice* whose name is what the design covers and whose content is either a fixed value or a way to draw one. If both are ever built, they should be one concept.

A deeper integration with `Supposition.jl` would be an interesting joint project. A covering design could supply the outer loop over classes, `Supposition` could supply the inner values, and `Supposition`’s shrinking could reduce a failure *within* its class while the design reports which classes it was in. That is well beyond the next release, but it is the kind of connection between packages the Julia community tends to value.

4.13 Getting cases out

- **Tables.** Free, once cases are `NamedTuples` (Fable).
- **Literal Julia code** (`as_code`, Opus). This is small and has unusually high value. It serves reviewability, since a committed test file shows every case in the diff, and it underlies the design lockfile. It also serves agents: generated code whose cases can be read is easier to trust.
- **GitHub Actions matrix** (Opus). The `include: list` is JSON. A documented recipe that uses any JSON package does the job without adding a dependency.
- **TOML** (Opus; your `feature/command-line` branch). Fable points out that the command-line tool cannot express constraints today at all, and it becomes complete only once constraints can be written as strings. That is an argument for the macro’s captured source, or for exact patterns, both of which serialize naturally.

Astra’s “no general serialization framework” and Opus’s exports are compatible: tables for free, one printer for literal code, and recipes for the rest.

4.14 Words

Vocabulary teaches. The names a package chooses become the concepts its users think with. Here is how the three proposals compare, with a suggestion for each concept. The first table covers nouns and keywords.

Concept	Today	Astra	Fable	Opus	Suggested
Space	none	TestSpace	ParameterSpace	TestSpace	TestSpace
Dimension	argument	factor	parameter	parameter	parameter
Rules	disallow	rules	constraints	forbid	constraints
Required cases	seeds	seeds	seeds	must_include	must_include
Strength	n_way	n_way	strength	strength	strength
Mixed strength	wayness	extra	stronger	stronger	stronger
Result	a vector	TestCases	TestCases	TestDesign	TestCases

The second covers functions. Today the general entry point is `all_tuples`, and the package has no functions for measurement, explanation, or preview.

Purpose	Astra	Fable	Opus	Suggested
General entry	<code>all_tuples</code>	<code>covering</code>	<code>all_pairs</code> and <code>kin</code>	<code>open question</code>
Measure	<code>coverage</code>	<code>coverage</code> , <code>missing_interactions</code>	<code>coverage</code> , <code>topup</code> , <code>extend</code>	<code>coverage</code>
Explain	<code>explain</code> , <code>isallowed</code>	<code>report</code>	<code>summary</code> , <code>diagnose</code>	<code>explain</code> , <code>report</code>
Preview	<code>none</code>	<code>design_sizes</code>	<code>compare_strengths</code>	<code>either name</code>

Some reasons. “Constraints” is the term used by the literature, by PICT, and by ACTS, so users who search for it will find the package. “Parameter” is the standard term in combinatorial testing; “factor” and “level” come from design of experiments, and “argument” is too narrow, since a dimension need not be a function argument. `must_include` avoids the collision with random seeds. `TestCases` says what the object is for, whereas “design” suggests design of experiments. For the general entry point, `covering` names the mathematical object and `all_tuples` has history, so it is a matter of taste. Keep the familiar `all_pairs` and `all_triples` either way, because those are the names people search for.

4.15 First contact: positioning and documentation

All three agree that the README should lead with the situation rather than the algorithm. Fable’s decision table is the best starting point. Here it is, lightly adapted:

Your situation	Reach for
A few representative values per parameter, several parameters, and bugs that plausibly live in <i>combinations</i> The same, but every run is cheap and the product is small	A covering design: <code>all_pairs</code> , <code>all_triples</code> <code>full_factorial</code> or <code>Iterators.product</code>
One known-good configuration, and a question about which single or paired changes break it Values you can generate but not enumerate, cheap runs, and a wish to see failures shrunk Hand-written cases already, and a question about what they miss	Excursions Property-based testing (<code>Supposition.jl</code>) coverage, then <code>must_include</code>
Both of the first and fourth rows	A design picks the class for each parameter; a generator draws within it

Opus adds three rows for when *not* to use the package. Use design of experiments to estimate how much each factor matters, space-filling samples to sweep continuous parameters, and fuzzing or property-based testing to hunt for the one value that breaks a routine.

On the manual, the proposals agree more than their outlines suggest. Fable’s levels (one line, then names, then constraints, then everything else, then inspection) make a tutorial. Opus’s jobs (test a function with many options, test generic code across types, plan a CI matrix, run a simulation campaign, audit an existing suite, diagnose a failure, test invalid inputs, combine with property-based testing) make a set of how-to guides. Both belong in the manual, and they fit the familiar split between tutorials, how-to guides, reference, and explanation. Two more points have full agreement:

- **Move the paper’s “How to Use” section into the manual.** Choosing equivalence classes, deciding how many values to list, and writing oracles are the most important human decisions in combinatorial testing, and today the paper is the only place that explains them well.
- **Make every example a doctest.** Opus notes that this alone would have caught the broken wayness example.

4.16 AI agents as users

Opus treats AI coding agents as a user group with specific needs: they must be able to tell from the docstrings when the package applies; they need results that describe themselves, determinism so that reruns match, errors that name the parameter and suggest a fix, a way to check claims (coverage), literal output that a human reviewer can read in a diff, and a one-page guide. Astra takes the opposite position explicitly: “Human users and AI assistants receive the same interface... No AI-specific API is required.”

On substance they agree. Everything Opus asks for is good design for human users too, and Astra’s interface already provides most of it: structured errors, named uncovered combinations, and ordinary Julia data. The only agent-specific items are a machine-readable summary (a `NamedTuple` that can be serialized) and a short guide, and the guide is documentation rather than API. These three documents, and this one, were written by models working in your repository, so the agent

user is not hypothetical. The most important thing for agents is also the most important thing for the humans who review their work: a claim that can be checked in one line.

4.17 What none of the three addresses

A synthesis should also note the gaps.

- **Oracles.** The package chooses inputs. It does not know the right answers. The paper’s conclusion calls the shift to checks that work for any combination of arguments “the most difficult part of the transition.” None of the proposals addresses it, even in documentation. The paper’s own advice (reference implementations, invariants, inverse problems, symmetries, and metamorphic relations) belongs in the manual next to the generation functions.
- **Weights and operational profiles.** Your Turing branch asks whether test generation could learn from recorded real usage or honor weights. None of the three touches this. It connects to prioritization and to budgets.
- **Sequences.** The paper suggests that parameters could be successive calls to a module’s functions. Kuhn and colleagues developed *sequence covering arrays* for exactly this. It remains untouched.
- **Validating the model.** The guarantee is relative to the chosen values. If two values of a parameter always take the same branches, one may be redundant; if a branch is never taken, a class may be missing. Measuring code coverage per case would validate the model, and the “Test selection” section of your methods page already describes why that is hard in Julia.
- **Stability by design.** Only the must-include workaround addresses stability under edits. An engine that deliberately minimizes change from a previous design is an open problem.

5 The larger context

5.1 What the package is for, underneath

Stripped to essentials, a combinatorial test suite makes a claim: *every combination of t of these choices was exercised, except these, which are impossible for these reasons*. The claim is only as good as the choices. The paper says so (“your tests are only as good as the argument values you choose”), and so does Astra (“It does not mean ... coverage of every program path”). Within that model, however, the claim is exact, checkable, and cheap to make.

This is why correctness is the product rather than a feature. A testing tool exists to give justified confidence. A tool that crashes is annoying, but a tool that silently drops a feasible pair and reports success is worse than no tool, because it creates confidence that is not justified. The over-forbidding example in [Where the package stands](#) is the failure to fear most. All three proposals are, at bottom, efforts to make the package’s claim true, complete, and visible. Astra insists that the claim be true. Fable makes it easy to state. Opus has the package say it out loud and helps the user act on it.

5.2 What these proposals teach about testing

You mentioned wanting to understand testing more deeply. The three proposals touch nearly every central idea in the field, often without naming it. Here is a map.

1. **Input modeling is the human part.** Choosing parameters and representative values is category-partition testing, and it is where both the skill and the risk lie. The generator only

makes the model’s consequences exact. A good next version should make the model visible, because a visible model can be reviewed.

2. **Constraints are a specification, and checking them finds bugs in the specification.** Implied exclusions are consequences of rules that nobody wrote down. Showing them is a small act of formal methods: the tool derives facts from a specification and asks whether the author meant them.
3. **Local and global consistency.** The failures in the current engines are a textbook case of a local check standing in for a global question. Building the feasibility and completability machinery yourself teaches more about constraint satisfaction than reading about it.
4. **Coverage criteria and their limits.** Interaction coverage is one adequacy criterion among many: line, branch, MC/DC, and mutation coverage are others. Inozemtseva and Holmes found that coverage is not strongly correlated with a suite’s effectiveness once suite size is controlled for. Interaction coverage is subject to the same caution. It measures the model, not the program.
5. **Random versus systematic testing.** This is an old debate, from Duran and Ntafos in 1984 to Arcuri and Briand in 2012, and it has a precise mathematical core, set out in the next section.
6. **The oracle problem.** Automated input generation is only as useful as automated judgment of outputs. Barr and colleagues surveyed the problem. Metamorphic testing, which checks relations between outputs such as $f(a, b) == f(2a, b/2)$, is the best-known partial answer, and the paper already recommends it.
7. **Diagnosis, masking, and adaptive testing.** What a test suite can tell you after a failure is a different question from what it covers, with its own mathematics (locating arrays, group testing) and its own pitfalls (masking).
8. **Negative testing and error masking.** Invalid inputs need their own design discipline, which category-partition understood in 1988.
9. **Reproducibility.** A test that cannot be rerun exactly cannot be debugged. Determinism and recorded seeds are a small instance of the broader problem of flaky tests.

Each option in [Options](#) engages a different subset of this list, and that is one of the ways to choose among them.

5.3 What the evidence does and does not show

It is worth being clear about what we actually know regarding the value of combinatorial testing, because it affects how the package should present itself.

The interaction rule. Kuhn, Wallace, and Gallo (2004) found that the failures they studied were triggered by at most six interacting parameters, and most by one or two. This is the empirical basis for covering designs, and it is an observation about particular systems, not a law.

Random at equal budget. Fable measured how random suites compare with designed ones:

Parameters $\times$ values	Designed pairwise cases	Random cases to cover every pair (median)	Share of pairs covered by that many random cases
5×4	16	84	64%
10×4	28	106	84%
10×8	113	530	83%

Parameters $\times$ values	Designed pairwise cases	Random cases to cover every pair (median)	Share of pairs covered by that many random cases
40×4	45	151	95%
40×8	166	709	93%

The last column follows from a short calculation. Suppose each parameter takes v values, and consider one particular t -way combination. A uniformly random case contains it with probability v^{-t} , so a random suite of N cases misses it with probability $(1 - v^{-t})^N$ and contains it with probability $1 - (1 - v^{-t})^N$. For ten four-valued parameters, pairwise, with $N = 28$, that is $1 - (15/16)^{28}$, about 84%. The formula reproduces every entry in Fable’s last column to the percent. It has two consequences:

- A covering array must contain all v^t combinations of any t columns, so it has at least v^t rows. A random suite of the same size therefore contains any *particular* combination with probability at least $1 - (1 - v^{-t})^{v^t}$, which is about $1 - 1/e$, or 63%. This is the heart of Arcuri and Briand’s argument that random testing is better at finding interaction faults than coverage figures alone suggest.
- The minimum size of a covering array grows only logarithmically with the number of parameters, so as parameters are added, a random suite of the same size covers a growing share of combinations. That is why random suites reach 93–95% at forty parameters.

So the honest pitch is not “an order of magnitude more faults.” At equal budget, a designed suite covers 100% of combinations where a random suite covers 64–95%, and the difference shrinks as parameters are added. Reaching the full guarantee with random cases takes four to six times as many runs, because the last few combinations are expensive to hit by chance. What designs buy is *the guarantee itself*: nothing is left to luck, you get an exact list of what was impossible, and extension is deterministic. That matters most when runs are expensive, when someone needs the claim (a reviewer, a funder, a CI budget), when parameters are few and have many values, and, as a hypothesis worth testing, when constraints are tight. Random sampling of valid configurations can make some feasible combinations rare, while a design treats every feasible combination equally.

What would count as evidence. Astra names it: authoring effort, execution cost, and fault detection on real code, against exhaustive and random selection at the same budget. Fault detection can be measured with *mutation analysis* (Jia and Harman surveyed the field): inject small faults into the code under test and count how many each suite catches. I know of no such study on Julia code. That is one reason the research track in [Options](#) matters.

5.4 Contributing to the Julia community

You described this work as a way to contribute to a community you appreciate. That framing changes some of the choices.

What Julia specifically needs. Opus makes the argument I find most compelling. In Julia, combinations of element type, container type, and algorithm really are different compiled methods, because of multiple dispatch. Generic code is the norm, and bugs live at particular method combinations: `Float32` with views, `BigFloat` with a particular factorization, an offset array with a specific algorithm. The interaction rule is especially apt for Julia. A guide titled something like

“testing generic code with covering designs” would be valued on its own merits, and it is the most natural way to introduce the package to a Julia audience.

Adoption is cheap and trust is expensive. The package is used only in tests, so adopting it adds no runtime dependency, and it already has only two dependencies of its own. Nothing about installing it is hard. What stops people is conceptual (when is this the right tool?) and a matter of trust (does it work with my rules?). The decision table addresses the first. The constraint fix and the coverage function address the second.

Stewardship is the contribution. Most Julia packages are maintained by one person, and every exported name is a promise. A small package that stays correct, documented, and compatible for years does more good than a large one that stalls. Astra’s “not now” list is a gift to the future maintainer, who is you. Version 1.0 in the Julia ecosystem means a SemVer commitment, and it is worth reaching deliberately. Before any breaking change, check which registered packages depend on `UnitTestDesign`, and consider raising the Julia compatibility floor, currently ~ 1.2 , to at least 1.9, where package extensions arrived. In practice that means the current long-term-support release. Extensions are how optional integrations (`Tables` display, `Supposition.jl`, a solver) can live alongside a small core.

Knowledge is also a contribution. Three things you now have are useful beyond this package:

- the finding that constraint handling is where combinatorial testing tools break, together with the 1,000-problem benchmark and the independent checker that demonstrate it (a JuliaCon talk on this would interest people well beyond Julia);
- the explanation of when designs beat random testing and when they do not, including the formula above;
- the manual’s guidance on equivalence classes and oracles, which applies to any kind of automated test generation.

Engagement. A release that fixes constraints is a good occasion for a Discourse post that asks for real test suites to try the package on. That recruits the case study the next version needs, and it gives the community a way to shape the named interface before it is frozen.

5.5 What you would learn from each path

Work	What it teaches about testing
Constraint core (feasibility, completability)	Constraint satisfaction; local versus global consistency; why tools fail at the edges of their specification
Named model and rules	Input modeling; category-partition; API design as a contract
Coverage measurement	Adequacy criteria and their limits; executed versus passed coverage
Diagnosis and follow-ups	Fault localization; masking; group testing; adaptive testing
Invalid values	Negative testing; error masking
Partitions with property-based testing	The partition-versus-random debate in practice; shrinking
A case study with mutation analysis	How to evaluate test effectiveness empirically, which is the hardest and most honest question in the field

6 Questions the best version must answer

These are the decisions that remain after the consensus. Each note gives where the proposals lean and, briefly, where I lean.

6.1 Purpose and audience

1. **Is this a covering-array generator with a better interface, or a toolkit for reasoning about test spaces?** All three move toward the second, Astra least and Opus most. I lean toward a toolkit with a small core: the model plus generate, measure, and explain.
2. **Whom should the next version delight first?** The candidates are the generic-code author (the most numerous in Julia, and working inside `runtests.jl`), the simulation developer (the paper's user, the one with the most constraints, and closest to your own background), the maintainer adopting gradually (served by `coverage`), and the CI engineer (served largely by recipes). I would pick the first and third, since the constraint fix already serves the second.
3. **What is the package's promise, in one sentence?** Three candidates, corresponding roughly to the three proposals:
 - "Give it the values you would loop over, and it returns far fewer cases that still exercise every pair."
 - "Describe the configurations your code must handle; it tells you which combinations your tests exercise, and supplies the fewest cases that cover the rest."
 - "It plans, generates, and diagnoses combinatorial tests, saying at each step what it promised and what remains."

The second is the one I would write at the top of the README.

6.2 The model

4. **What belongs in a `TestSpace`?** The candidates are parameters, values, and constraints (consensus), plus possibly value labels, invalid markers, partitions, and weights. My lean: the consensus three now, with labels and partitions merged into one concept if they are ever built.
5. **Where do constraints live?** In the space (Astra, Fable), or in the call (Opus)? My lean: the space, with a convenience keyword on one-shot calls.
6. **How are dependent parameters expressed?** Through the sentinel recipe now, and perhaps first-class conditional parameters later.

6.3 Constraints

7. **Which rule forms ship first, and when does the macro come, if ever?** My lean: exact patterns and functions with listed names first, both compiling to one internal rule type, with the macro later if users ask for it and it becomes needed for serialization.
8. **Are whole-case predicates allowed, and how is their cost exposed?** Astra allows them as an escape hatch. My lean: allow them, document their cost, and warn when feasibility checking becomes slow.
9. **How far should the package go in attributing exclusions to rules?** Name the rule for direct exclusions, run a simple deletion search for implied ones, and never invent causes.

10. **What happens to legacy `disallow`?** Treat it as a pruning oracle during the correctness release, then deprecate it in favor of scoped rules, or evaluate it on complete cases only.

6.4 Results

11. **Should `TestCases` subtype `AbstractVector` with concrete element types, and should positional calls change their return type at 1.0?** My lean: yes to the first. Decide the second after checking the package’s dependents.
12. **What may show `compute`?** Only bookkeeping from generation. Leave verification, bonus coverage, and prefix curves to explicit calls.
13. **What exactly is promised about determinism?** Across runs, yes. Across versions, no; that is Astra’s boundary. Across edits of the space, only through must-include cases.
14. **Should designs be generated when the tests run, or committed as data?** Both should be supported, and the documentation should present the design lockfile as the practice for expensive or reviewed suites.

6.5 Scope

15. **Does the package ever see test outcomes?** If it does, my lean is to take them as a pure function of cases and outcomes, with no runner.
16. **Negative testing: Invalid values, a second space, or both?** A second space documented now, and Invalid values later.
17. **Property-based testing: a documented pattern, a `Partition` type, or an extension?** A pattern first, and an extension if a `Supposition.jl` collaboration develops.
18. **Which export formats are core?** Tables (free) and literal code in the core; everything else as recipes.
19. **Z3 or SAT, ever?** Only as an extension, and only if real spaces appear that backtracking cannot handle.

6.6 Evidence and stewardship

20. **What evidence should come before 1.0?** A case study on at least one real Julia test suite, measuring authoring effort, cost, and fault detection (Astra’s standard).
21. **What should the Julia compatibility floor be?** At least 1.9, for package extensions.
22. **What will you say no to, and where will that be written?** A “non-goals” section in the manual makes the boundary public, and it protects contributors and you alike.
23. **The names.** `TestSpace` or `ParameterSpace`; constraints or rules; `must_include` or `seeds`; covering or `all_tuples`; `TestCases` or `TestDesign`. My suggestions are in the vocabulary table in [Words](#).

7 A portrait of the best version

Before the options, it may help to picture what using a good next version would feel like. The example below is illustrative; its numbers are invented.

A maintainer of a generic linear-algebra package has a test file that loops over three element types and two array types for each of four algorithms. Someone has just reported a failure with `Float32`

views and a pivoted factorization, a combination the loops never reached, because half of them were commented out when the suite became slow.

The maintainer writes down the space: element type, array type, algorithm, pivoting, and matrix size class. Pivoting applies only to two of the algorithms, so they add one `require` rule with a sentinel `:none`. Then they run `coverage` on the configurations their existing tests already use. The report says the tests cover 58 of 81 feasible pairs, and it lists the 23 that are missing by name, including `(eltype = Float32, storage = :view)`. It also notes that `(algorithm = :cholesky, pivot = :partial)` is excluded because the rule forbids it.

They call `all_pairs(space; must_include = existing)`. The existing cases come back first, followed by seven new ones. One of the new cases fails. They pass the outcomes to `diagnose`, which names `(eltype = Float32, storage = :view)` as the prime suspect and proposes two follow-up cases that separate it from a masked pair. The follow-ups confirm it.

They commit the cases with `as_code`, together with one test asserting that the committed cases still cover the space. Six months later a contributor adds `BigFloat`, that test fails and lists the new pairs, and regeneration keeps every old case and adds five.

Nothing in this story requires a runner, a macro, a solver, or a file format. It uses one noun, four verbs (`coverage`, `all_pairs`, `diagnose`, `as_code`), one rule form, and `must-include` cases.

8 Options for moving forward

Each option below is coherent on its own terms: it ships something useful, and it could be the stopping point. They are ordered roughly by scope.

8.1 Option 1: Repair in place

What ships (0.4.x or 0.5). The GND fix from your branch. A fixed default seed for `GND()`. A feasibility pass, the completability invariant, and guaranteed progress in both engines, all driven by legacy `disallow` used as a pruning oracle. Infeasible combinations reported through a warning that lists them. Validation with messages in the user's vocabulary for `must-include` cases, strength, and wayness. `wayness` copied rather than mutated, with ranges accepted. Single-valued parameters allowed. A size guard on `full_factorial`. A report when an excursion's base is forbidden. Opus's independent checker and random constrained problems added to the test suite. The engines page corrected, and examples turned into doctests.

Who it serves. Every current user, and above all those with rules.

Effort. Two to three weeks. Opus estimated its Phase 0 at one to two.

Risks. The `nothing` ambiguity and the over-forbidding trap remain, though they can be documented more prominently. The positional interface remains. Nothing new draws in new users.

What you would learn. Constraint satisfaction from the inside, and how IPOG and AETG-style engines behave under constraints.

Contribution. It turns a package that crashes on roughly one in nine small constrained problems into one that does not. For a testing tool, that is the most important single contribution available.

8.2 Option 2: A small named core (Astra-led)

What ships. Option 1, plus: `TestSpace` built from a `NamedTuple` or from pairs. Constraints as `forbid(pattern), require(names) do ... end`, and a whole-case `require`, all compiled to one tabulated internal rule. `TestCases` `<: AbstractVector` of `NamedTuples`, carrying its space and request, with a short display. `coverage(cases, space)` and `coverage(cases)`. `isallowed` and `explain`. `must_include` validated by name. `strength` and `stronger` by name. Excursions by name with an explicit base. The legacy positional API unchanged. A documentation rewrite with the decision table, a tutorial by levels, and the paper’s guidance on modeling and oracles.

Who it serves. Generic-code authors, simulation developers, maintainers and auditors, and agents, who get checkable claims.

Effort. One to two months beyond Option 1. Opus estimated two to four weeks for the equivalent phase, but the documentation is real work.

Risks. It is correct but quiet. Without planning tools or a rich display, adoption depends on the documentation and on examples.

What you would learn. API design as a contract; the semantics of coverage; input modeling.

Contribution. A small, well-specified tool that people can depend on without fear.

8.3 Option 3: The friendly product (Fable-led)

What ships. Option 2, plus: the `@forbid` and `@require` macros with printable source. Typed `TestCases{T}` with a table display and the “10 of 81” line. `covering(space; strength, stronger)` as the general entry point. Partial `must_include` cases. A planning function (`design_sizes` or `compare_strengths`). `missing_interactions`. `GND(candidates = ...)`. Version 1.0, with the positional return type changed and deprecations in place.

Who it serves. First-time users, and discovery through documentation and the REPL.

Effort. About a month beyond Option 2.

Risks. The macro is a small language to document, test, and maintain. Changing the positional return type breaks existing users. More surface has to be frozen at 1.0.

What you would learn. Julia metaprogramming, product design, and the practice of migrating users.

Contribution. A combinatorial-testing tool that is approachable and feels native to Julia.

8.4 Option 4: The testing workbench (Opus-led)

What ships. Option 3, plus: `run_cases` and outcome recording; `diagnose` and `followups`; `extend` and `topup`; `max_cases` and ordering for coverage; bonus-coverage reports; `Invalid` values; `Partition` values; `github_matrix`; TOML reading and writing; `as_code`; the agent guide; rolling coverage across CI runs.

Who it serves. All eight of Opus’s personas.

Effort. Several months. Opus’s first three phases add up to roughly six to ten weeks of focused work, and its fourth phase is ongoing.

Risks. Scope. The package drifts toward being a framework. Every new feature needs careful semantics of its own (diagnosis with several faults or intermittent failures, the coverage meaning of `Invalid` values, reproducibility of `Partition` draws). The maintenance burden grows, and freezing an API this large at 1.0 is hard.

What you would learn. The most: fault localization, masking, negative testing, and CI engineering.

Contribution. Potentially a distinctive tool; I know of no Julia package that diagnoses failures from covering designs. The risk is a set of half-finished features.

8.5 Option 5: Core and satellites (the synthesis)

What ships, in stages.

1. *Correctness release (0.5)*: Option 1.
2. *Named core (0.6)*: Option 2, plus three inexpensive pieces of Option 3: the “of N combinations” line in the display, partial must-include cases, and a planning function. Raise the Julia compatibility floor.
3. *Case study* (see the research track below), before more API is added.
4. *Inspection and diagnosis (0.7)*: `diagnose(cases, passed)` and follow-up cases as pure functions, in a module marked experimental; `as_code`; a report for bonus coverage and prefix curves.
5. *Satellites, as demand appears*: package extensions for `Supposition.jl` and possibly a solver; recipes for GitHub matrices and TOML (reviving the command-line branch once rules serialize); the macro, if users ask for it; `Invalid` values after diagnosis has settled.
6. *1.0*, after the case study and a period of use, with the decision on the positional return type made then.

Who it serves. Everyone Option 2 serves, then the debugger, then others as demand shows.

Effort. About the same total as Options 3 and 4, but spread across releases, each of which is useful by itself.

Risks. It depends on repeatedly saying no, or “not yet.” An experimental module has to be clearly labeled so that it does not become a promise by accident.

What you would learn. Both the discipline of contracts and the depth of diagnosis, with a case study between them to keep the work honest.

Contribution. A dependable core, room to explore, and published evidence.

8.6 A research track, alongside any option

Choose two or three real Julia test suites of different kinds: generic numerical code, a simulation configuration, and a CI matrix. For each, express the space; measure what the existing tests cover; complete it; seed faults (hand-written mutants will do if no tool fits); and compare exhaustive, random, pairwise, and three-way selection at equal budgets. Record how long authoring the space took. Write the results up as a JuliaCon talk or proceedings paper, a Discourse post, and a case study in the manual. Separately, publish the 1,000-problem constrained benchmark and the independent checker as regression tests that other implementers of combinatorial testing could reuse.

This track is where the question of whether the package is worth adopting gets answered, and where you would learn the most about testing itself.

8.7 The options side by side

	Option 1	Option 2	Option 3	Option 4	Option 5
Constraint fix	Yes	Yes	Yes	Yes	Yes
Names and TestSpace	No	Yes	Yes	Yes	Yes
coverage and explain	Partial	Yes	Yes	Yes	Yes
Macros	No	No	Yes	Yes	If asked
Planning function	No	No	Yes	Yes	Yes
Diagnosis	No	No	No	Yes	Experimental
Runner or outcome files	No	No	No	Yes	No
Exports beyond tables	No	No	No	Yes	Literal code; recipes
Invalid and Partition	No	No	No	Yes	Later
Breaking change	No	No	Yes	Yes	Decided at 1.0
Effort	Weeks	Months	Months	Many months	Staged

9 A recommendation

I recommend Option 5, with the research track. Here are the decisions I would make now, and those I would defer.

9.1 Decide now

1. **Merge the GND branch** for what it fixes, and make its commit message precise: it fixes scoring and bounds the candidate loop, and it does not fix constraints. Record in an issue that the implicit-constraint hang remains, so that the correctness release closes that issue.
2. **Adopt Astra’s semantic contract as the written specification**, preferably in the developer documentation, and judge every later feature against it. The six points in [The semantic contract](#) are a good draft.
3. **Build the constraint core under the legacy API first**: a feasibility pass using legacy `disallow` as a pruning oracle, the completability invariant in IPOG’s three placement sites and in GND’s candidate construction, progress guarantees, and final validation. Add the independent checker and the random constrained problems to the test suite on the first day, so that they drive the work. Release 0.5.
4. **Fix the internal rule representation**: a scope, a predicate, a polarity, and a label, tabulated over the scope. Every surface syntax, including legacy `disallow`, becomes a constructor for this type.

5. **Build the named core with two rule forms**, exact patterns and functions with listed names. Defer the macro. Keep positional return types unchanged until 1.0.
6. **Export coverage as early as possible**. It is the cheapest high-value feature and the way into existing projects.
7. **Do the case study before adding more API**. It will produce real failures, which is the right material on which to design diagnose.

9.2 Defer, with reasons

- **The macro**, until users ask for it, or until the command-line format needs rules as strings.
- **Invalid values**, until diagnosis has settled, since both concern how failures interact with coverage.
- **Partition**, until the documented pattern proves insufficient.
- **A runner and outcome files**, possibly forever. A pure `diagnose` function covers the need.
- **The command line, TOML, and GitHub matrices**, as recipes after rules can be serialized.
- **Z3 or SAT**, until a real space defeats backtracking.
- **Rolling coverage**, until outcome recording exists for some other reason.

9.3 The first three weeks

- **Week 1**. Merge the GND branch. Write the independent checker and the random-problem generator as tests; they will fail, and that is the point. Write the feasibility pass.
- **Week 2**. Add the completability invariant to IPOG and GND. Add the progress guarantees and the validation messages. Turn the failing tests green.
- **Week 3**. Fix the smaller items (`wayness`, single-valued parameters, `full_factorial`, excursion reports). Correct the engines page, add doctests, and put the decision table at the top of the README. Release 0.5, and announce it on Discourse with a request for real test suites to try it on.

After that, the named core and the case study can proceed in parallel. The case study will tell you which parts of Options 3 and 4 people actually need.

9.4 A closing thought

The three documents disagree about scope, syntax, and ambition, but they share a single conviction: the package should tell the truth about what it did. Astra wants the truth precise, Fable wants it easy to see, and Opus wants it spoken at every step. That conviction is also the heart of testing. A test suite is a claim about what was checked, and it is worth exactly as much as that claim can be trusted. Building a tool whose claims can be trusted, making them easy to check, and showing others in the Julia community how to use them well would be a real contribution, and a good way to understand testing deeply.

Appendix A: One example in four dialects

The running example from Fable: a solver with a mode, a solver choice, and a tolerance. There are two rules. Fast mode uses no solver, and exact mode needs a tight tolerance. Together they

make (solver = :lu, tol = 1e-3) and (solver = :qr, tol = 1e-3) impossible, though neither rule mentions both parameters.

Today. The rule has to tolerate nothing, and the call crashes. I confirmed the crash on the current branch.

```
mode    = [:fast, :exact]
solver  = [:none, :lu, :qr]
tol     = [1e-3, 1e-6]
function dis(m, s, t)
    fast_with_solver = m == :fast && s !== nothing && s != :none
    exact_loose      = m == :exact && t !== nothing && t > 1e-4
    fast_with_solver || exact_loose
end
all_pairs(mode, solver, tol; disallow = dis)
# BoundsError: attempt to access 2-element Vector{Symbol} at index [0]
```

Astra.

```
space = TestSpace(
    (mode = (:fast, :exact),
     solver = (:none, :lu, :qr),
     tol = (1e-3, 1e-6));
    rules = [
        require((:mode, :solver);
            reason = "only exact mode uses a solver") do m, s
            m == :exact || s == :none
        end,
        forbid((mode = :exact, tol = 1e-3);
            reason = "exact mode needs a tight tolerance"),
    ],
)
cases = all_pairs(space)
explain(space, (solver = :lu, tol = 1e-3)) # infeasible
coverage(cases)
```

Fable.

```
space = ParameterSpace(
    :mode => [:fast, :exact],
    :solver => [:none, :lu, :qr],
    :tol => [1e-3, 1e-6];
    constraints = [
        @forbid(mode == :fast && solver != :none),
        @require(mode == :fast || tol < 1e-4),
    ],
)
cases = all_pairs(space)
missing_interactions(handwritten, space)
```

Opus.

```

space = TestSpace(
  mode    = [:fast, :exact],
  solver  = [:none, :lu, :qr],
  tol     = [1e-3, 1e-6],
)
design = all_pairs(space; forbid = [
  (mode, solver) -> mode == :fast && solver != :none,
  (mode, tol)    -> mode == :exact && tol > 1e-4,
])
coverage(design)

```

A synthesis.

```

space = TestSpace(
  (mode = [:fast, :exact],
   solver = [:none, :lu, :qr],
   tol = [1e-3, 1e-6]);
  constraints = [
    require(:mode, :solver;
      reason = "only exact mode uses a solver") do m, s
      m == :exact || s == :none
    end,
    forbid((mode = :exact, tol = 1e-3);
      reason = "exact mode needs a tight tolerance"),
  ],
)
cases = all_pairs(space)

```

The display would read as follows. The figures are those of Fable's prototype.

```

TestCases: 5 cases, strength 2 (IPOG)
  space      3 parameters; 12 combinations, 5 valid
  covers     all 11 feasible pairs
  excluded   3 pairs forbidden by constraints
              2 pairs impossible because constraints combine:
                (solver = :lu, tol = 0.001)
                (solver = :qr, tol = 0.001)
  (mode = :fast, solver = :none, tol = 0.001)
  (mode = :exact, solver = :none, tol = 1.0e-6)
  (mode = :exact, solver = :lu, tol = 1.0e-6)
  (mode = :exact, solver = :qr, tol = 1.0e-6)
  (mode = :fast, solver = :none, tol = 1.0e-6)

```

The counts check out. There are 16 pairs in all: 6 for mode and solver, 4 for mode and tolerance, and 6 for solver and tolerance. Three are forbidden directly and two by implication, which leaves 11 feasible pairs. In this tiny space the pairwise design happens to be the entire valid space, which is itself worth reporting.

Then come the everyday follow-ups:

```

explain(space, (solver = :lu, tol = 1e-3))

```

```
coverage(handwritten, space)
all_pairs(space; must_include = handwritten)
all_triples(space; must_include = cases)
```

Appendix B: Glossary

Covering array. A set of cases such that, for every choice of t parameters, every combination of their values appears in at least one case. Here t is the *strength*; pairwise means $t = 2$.

Mixed strength. A higher strength required for a named subset of parameters, on top of the base strength.

Input parameter model. The tester's chosen parameters, representative values, and constraints. The object called `TestSpace` in this document.

Category-partition method. Ostrand and Balcer's 1988 method for deriving tests from categories, choices, and constraints, with `[error]` and `[single]` annotations for choices tested only once.

Feasible combination. A combination contained in at least one complete case that satisfies every constraint.

Implicit (implied) constraint. A combination made infeasible by several rules together, though no single rule forbids it.

Greedy dead end. A partly built row that no allowed value can complete, even though every required combination is feasible.

Completeness. The property that a partial case can still be extended into a valid complete case.

Excursion. A case that differs from a base case in at most d parameters.

Must-include case (seed). A case, possibly partial, that has to appear in the result, and whose combinations count toward coverage.

Interaction coverage. The share of feasible t -way combinations that appear in a set of cases. Distinct from line or branch coverage.

Masking. The effect by which a failing case hides the behavior of the other combinations it contains, so that they remain unverified.

Locating array. A covering array whose pass and fail pattern identifies a small number of faulty interactions.

Oracle. The means of deciding whether a test's output is correct.

Metamorphic relation. A relation that must hold between the outputs of related inputs, used as an oracle when the correct output is unknown.

Mutation analysis. Measuring a suite's effectiveness by injecting small faults into the code and counting how many the suite detects.

Appendix C: References

- Arcuri, A., and L. Briand. "Formal analysis of the probability of interaction fault detection using random testing." *IEEE Transactions on Software Engineering* 38(5), 2012.
- Barr, E. T., M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. "The oracle problem in software testing: A survey." *IEEE Transactions on Software Engineering* 41(5), 2015.
- Cohen, D. M., S. R. Dalal, M. L. Fredman, and G. C. Patton. "The AETG system: An approach to testing based on combinatorial design." *IEEE Transactions on Software Engineering* 23(7), 1997.
- Cohen, M. B., M. B. Dwyer, and J. Shi. "Constructing interaction test suites for highly-configurable systems in the presence of constraints: A greedy approach." *IEEE Transactions on Software Engineering* 34(5), 2008.
- Colbourn, C. J., and D. W. McClary. "Locating and detecting arrays for interaction faults." *Journal of Combinatorial Optimization* 15(1), 2008.
- Czerwinka, J. "Pairwise testing in the real world: Practical extensions to test-case scenarios." *Pacific Northwest Software Quality Conference*, 2006. (PICT.)
- Duran, J. W., and S. C. Ntafos. "An evaluation of random testing." *IEEE Transactions on Software Engineering* SE-10(4), 1984.
- Ghandehari, L. S., et al. "A combinatorial testing-based approach to fault localization." *IEEE Transactions on Software Engineering* 46(6), 2020.
- Grindal, M., and J. Offutt. "Input parameter modelling for combination strategies." *Proceedings of the IASTED International Conference on Software Engineering*, 2007.
- Grindal, M., J. Offutt, and S. F. Andler. "Combination testing strategies: A survey." *Software Testing, Verification and Reliability* 15(3), 2005.
- Inozemtseva, L., and R. Holmes. "Coverage is not strongly correlated with test suite effectiveness." *International Conference on Software Engineering*, 2014.
- Jia, Y., and M. Harman. "An analysis and survey of the development of mutation testing." *IEEE Transactions on Software Engineering* 37(5), 2011.
- Kuhn, D. R., R. N. Kacker, and Y. Lei. *Introduction to Combinatorial Testing*. CRC Press, 2013.
- Kuhn, D. R., D. R. Wallace, and A. M. Gallo. "Software fault interactions and implications for software testing." *IEEE Transactions on Software Engineering* 30(6), 2004.
- Kuhn, D. R., I. Dominguez Mendoza, R. N. Kacker, and Y. Lei. "Combinatorial coverage measurement concepts and applications." *Second International Workshop on Combinatorial Testing*, 2013. (NIST CCM.)
- Lei, Y., R. Kacker, D. R. Kuhn, V. Okun, and J. Lawrence. "IPOG/IPOG-D: Efficient test generation for multi-way combinatorial testing." *Software Testing, Verification and Reliability* 18(3), 2008.
- Nie, C., and H. Leung. "A survey of combinatorial testing." *ACM Computing Surveys* 43(2), 2011.
- Ostrand, T. J., and M. J. Balcer. "The category-partition method for specifying and generating functional tests." *Communications of the ACM* 31(6), 1988.

Petke, J., M. B. Cohen, M. Harman, and S. Yoo. “Practical combinatorial interaction testing: Empirical findings on efficiency and early fault detection.” *IEEE Transactions on Software Engineering* 41(9), 2015.

Segura, S., G. Fraser, A. B. Sanchez, and A. Ruiz-Cortes. “A survey on metamorphic testing.” *IEEE Transactions on Software Engineering* 42(9), 2016.

Yilmaz, C., E. Dumlu, M. B. Cohen, and A. Porter. “Reducing masking effects in combinatorial interaction testing: A feedback driven adaptive approach.” *IEEE Transactions on Software Engineering* 40(1), 2014.

Yu, L., Y. Lei, M. Nourozborazjany, R. N. Kacker, and D. R. Kuhn. “An efficient algorithm for constraint handling in combinatorial test generation.” *IEEE International Conference on Software Testing, Verification and Validation*, 2013. (IPOG-C.)

Yu, L., Y. Lei, R. N. Kacker, and D. R. Kuhn. “Constraint handling in combinatorial test generation using forbidden tuples.” *IEEE International Conference on Software Testing, Verification and Validation Workshops*, 2015.

Supposition.jl: property-based testing for Julia. <https://github.com/Seelengrab/Supposition.jl>

The three source documents: `interface_astra.md`, `interface_fable.md`, and `interface_opus.tex`, in this repository.